

GPU-Parallel Multi-Entry Descent for HNSW: Entry-Point Quality as a Lever for Recall

Computing for Data Science 2 — Class Project Proposal

Kyungmin Kim (김경민) | May 17, 2026

0. Korean Abstract

VectorDB 에서 ANN 을 지원하는 인덱스들 중 de facto 인 HNSW 는 계층적 그래프 구조를 유지하며, 하나의 쿼리를 수행할 때 하나의 진입점을 고정해서 사용합니다. 다만 그 진입점과 찾아야하는 query 간의 거리가 탐색의 난이도에 영향을 주기 때문에 진입점은 고정해서 사용하면 최적의 결과를 얻기 힘든 상황이 발생합니다. 이에, 하나의 쿼리에 대해 여러개의 진입점을 모두 수행해보고 가장 좋은 recall 값을 가지는 결과를 뽑는 접근을 해보려고 하며, 그 과정에서의 연산 속도 향상을 위해 GPU 병렬 처리를 도입해보고자 합니다.

1. Problem Statement and Motivation

Hierarchical Navigable Small World (HNSW) [1], the dominant CPU graph-based approximate nearest neighbor (ANN) index, performs search as a sequence of greedy descents through hierarchical layers: a fixed entry point at the topmost layer is used to greedily navigate toward the query, the closest node found is handed off as the entry to the next layer down, and the process terminates with a bounded beam search at the base layer (layer 0). The base-layer beam search is where the recall budget is spent; the upper-layer descent serves only to produce its starting point.

In our concurrent CPU work, we observed a strong correlation between *the distance from the base-layer entry point (EP) to the query* and *query difficulty*: easy queries arrive at the base layer already close to their nearest neighbors, while hard queries arrive far away. This observation has a direct algorithmic consequence: **the upper-layer descent silently sets the budget required by the base-layer search**. A good EP makes the rest of the search cheap; a bad EP cannot be recovered by any reasonable base-layer ef.

Standard HNSW commits to a single descent from a single fixed top-layer entry, accepting whatever EP that descent produces. We propose to instead run K *independent* greedy descents from K distinct top-layer seeds and take their best (or all K) as the base-layer starting set. Because each descent is an independent draw from an upper-layer distribution, the expected distance from the closest-of- K EPs to the query decreases monotonically in K . The intervention targets exactly the EP-quality signal that we have measured to drive difficulty.

The K descents share three properties that make them an unusually GPU-friendly workload: 1) they are mutually independent, 2) they traverse the very small upper layers (which fit in shared memory), 3) and they follow identical control flow. **Each descent maps naturally to one warp or one thread within a single block, making K -way descent a rare case of natural data parallelism in graph search.** The combination of search-time multi-entry seeding on HNSW upper layers, parallelized on GPU, motivated by an empirical EP-quality observation has not, to our knowledge, been characterized in the literature.

2. Background and Related Work

Standard HNSW [1] uses a single fixed top-layer entry. iQAN [4] introduces intra-query parallelism on CPUs by dispatching multiple threads from different entries, but focuses on the base layer and the multi-core regime. HNSW++ [2] partitions the dataset into two branches with independent hierarchies at build time, reporting +18–30% recall on NLP and CV datasets; this is an index-time intervention rather than a search-time one and requires storing two indexes. FlatNav [3] argues that hierarchical layers are largely redundant in high-dimensional regimes because of an emergent 'hub highway' that subsumes the navigational role of the hierarchy. This Hub Highway Hypothesis is a natural source of skepticism for any entry-diversification proposal: if all descents converge to the same hub neighborhood, multiple entries should buy little. Quantifying that absorption effect empirically is part of Q1 below. On the GPU side, CAGRA [5] adopts a flat graph with multi-CTA cooperation, addressing GPU utilization but not single-entry bias at the algorithmic level.

3. Research Questions

- **Q1a. Algorithmic EP-quality improvement.** As K varies over $\{1, 2, 4, 8, 16 \dots \max\}$, how does the expected distance from the closest-of- K candidate entry points to the query decrease? This is a directly measurable algorithmic quantity that should decrease monotonically with diminishing returns; the empirical question is the shape of that curve.

- **Q1b. Downstream recall gain.** Does the EP-quality improvement translate into recall gains at fixed base-layer ef? Or does the Hub Highway effect [3] absorb the improvement, leaving downstream performance unchanged? A null result here is itself an informative empirical confirmation of the Hub Highway Hypothesis for HNSW.
- **Q2. Entry selection strategy.** How should the K seeds be chosen? Three candidates: (i) uniform random over top-layer nodes, (ii) farthest-point sampling for explicit diversity, (iii) cluster-anchor seeds from a coarse k-means over upper-layer nodes. The hypothesis is that diversity-aware selection extracts more EP-quality improvement per additional K.
- **Q3. GPU cost of K-way descent.** How does end-to-end latency scale with K when descents are run in parallel on GPU versus serially on CPU? The hypothesis is that K-way descent is nearly free on GPU (K warps within one block) until shared-memory or register pressure binds, while CPU cost grows roughly linearly. Deliverable: a Pareto curve of (recall, latency) across K for both implementations.

4. Parallelization and Optimization Strategy

- **Parallelism mapping:** 1 thread block = 1 query. Within a block, K warps (or K threads when $K \leq 32$) each execute one independent greedy descent through the upper layers. Descents share read-only access to the upper-layer adjacency lists, which are small (HNSW upper layers contain $O(N / M^{\text{level}})$ nodes) and fit in shared memory for the datasets considered.
- **CUDA features exercised:** warp-shuffle reductions for distance computation, shared memory for upper-layer adjacency and per-descent visited sets, coalesced reads of query and neighbor vectors, and divergent-tolerant control flow (descents terminate at different steps, but cost is dominated by distance compute, not branching).
- **Base-layer handoff:** after K descents terminate, the K candidate entry nodes are deduplicated and passed as the initial frontier to a standard base-layer beam search. The base-layer search itself is unchanged.
- **Baselines:** (i) standard HNSW search ($K=1$, CPU hnsplib), (ii) CPU multi-entry descent (sequential K iterations) to separate algorithmic benefit from GPU benefit, (iii) GPU $K=1$ descent to isolate raw GPU overhead.
- **Datasets:** GloVe-100 ($1.2M \times 100$; classical hard dataset where EP-quality variance is high) and agnews-mxbai-1024 ($\sim 770K \times 1024$; modern LLM-era embedding evaluated at recall target 0.99).

5. Task Breakdown and Timeline

Week	Dates	Tasks and Deliverables
1	May 18 – May 24	Fork hnsplib; implement CPU K-way multi-entry descent with random seeding; early validation on GloVe-100—measure Q1a (best-of-K EP distance vs K) and Q1b (recall vs K at fixed ef). Decision point: if Q1b shows no measurable gain at $K=4$, pivot to the multi-CTA measurement plan held as fallback.
2	May 25 – May 31	Implement GPU K-way upper-layer descent kernel: 1 block per query, K warps per descent, shared-memory upper-layer adjacency. Verify GPU-CPU recall parity at $K \in \{1, 4, 16\}$.
3	Jun 1 – Jun 7	Full Q1 sweep on both datasets (EP distance + recall Pareto). Q2: entry selection strategy comparison (random / farthest / cluster). Q3: latency scaling with K on GPU vs CPU; Nsight Compute profiling for occupancy and warp efficiency.
4	Jun 8 – Jun 14	Final presentation (Jun 9); figure preparation; final report drafting and submission (Jun 14).

References

- [1] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE TPAMI, 42(4):824–836, 2018.
- [2] T. Nguyen et al. HNSW++: dual-branch hierarchical navigable small-world graphs for improved recall. arXiv preprint, 2025.
- [3] B. Munyampirwa, V. Lakshman, and B. Coleman. Down with the hierarchy: the 'H' in HNSW stands for 'hubs'. arXiv preprint, 2024.
- [4] Z. Peng, M. Zhang, K. Li, R. Jin, and B. Ren. iQAN: fast and accurate vector search with efficient intra-query parallelism on multi-core architectures. In PPOPP, pages 313–328, 2023.
- [5] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, and Y. Wang. CAGRA: highly parallel graph construction and approximate nearest neighbor search for GPUs. In ICDE, 2024.
- [6] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. IEEE TBD, 7(3):535–547, 2021.
- [7] NVIDIA Corporation. Nsight Compute Documentation. <https://docs.nvidia.com/nsight-compute/>, accessed May 2026.